

INSTITUTO TECNOLÓGICO DE CULIACAN

INGENIERIA EN SISTEMAS COMPUTACIONALES

Construcción de un modelo de Machine Learning para
imputación

CIENCIA DE DATOS

DENZEL ARTURO GUTIERREZ CHAVEZ

JUAN CARLOS QUIÑONEZ MADRID



TECNOLÓGICO
NACIONAL DE MÉXICO



CULIACAN, SINALOA

30/05/26

INSTRUCCIONES

1. Analiza detalladamente el dataset hidrometeorológico proporcionado, identificando los valores faltantes en las variables principales: precipitación, temperatura máxima, temperatura mínima y evaporación. Observa patrones de ausencia y considera cómo estos pueden afectar la calidad del análisis y la posterior imputación de datos.
2. Selecciona una metodología preestablecida para la construcción de un modelo de Machine Learning orientado a la imputación de datos. Describe con profundidad los fundamentos de la metodología elegida, explicando por qué resulta adecuada para este tipo de variables hidrometeorológicas y cómo se adapta al contexto del dataset. Incluye una breve comparación con otras metodologías alternativas, resaltando ventajas y limitaciones.
3. Procede a construir el modelo de Machine Learning para generar datos sintéticos que permitan completar los valores faltantes en el dataset. Detalla cada paso del proceso, desde la preparación de los datos, la configuración de hiperparámetros, hasta la generación de los valores imputados. Explica cómo el modelo aprende a estimar los datos y qué técnicas específicas se emplean para asegurar la precisión de la imputación.
4. Valida el desempeño del modelo utilizando métricas apropiadas para modelos de regresión, tales como el error cuadrático medio (MSE) y el coeficiente de determinación (R^2). Justifica la elección de estas métricas y analiza los resultados obtenidos, interpretando su significado en el contexto de la imputación de datos hidrometeorológicos. Considera la posibilidad de aplicar otras métricas complementarias si aportan valor al análisis.
5. Documenta y explica cada etapa realizada en el proceso, desde el análisis inicial hasta la validación del modelo. Asegúrate de que el informe incluya una descripción detallada de las decisiones tomadas, las razones detrás de cada elección metodológica, y los resultados obtenidos. El informe debe reflejar una comprensión profunda del proceso de imputación mediante Machine Learning y demostrar la capacidad de justificar cada paso con argumentos sólidos y basados en evidencia.

En el proyecto que toca en esta unidad, decidimos usar el mismo dataset que utilizamos en el Modulo 2 de limpieza de datos, el cual es un dataset hidrometeorológico con datos de temperaturas relacionadas a 6 décadas de información de la estación.

El proyecto se dividió por 6 fases, donde se toman en cuenta cada parte del proceso de construcción y entrenamiento del Modelo de Machine Learning

FASE 1 – CONOCER NUESTROS DATOS

En esta fase simplemente entramos a analizar todo lo necesario que debemos saber de nuestro dataset, como cuantas filas de datos existen dentro de este mismo, cual es el nombre de las columnas a trabajar, cuantos nulos hay por cada columna, y en donde exactamente se encuentran ubicados para proceder a su preparación de datos.

FECHA	0	2922	1969-01-01	decada	
PRECIP	40	3345	1970-02-28	1960	1
EVAP	272	11284	1992-05-26	1970	1
TMAX	5	12866	1999-04-17	1990	2
TMIN	6	14245	2003-01-31	2000	1
dtype: int64		Name: FECHA, dtype: datetime64[us]		dtype: int64	

FASE 2 - PREPARACION DE LOS DATOS

Conociendo las décadas y bajo las instrucciones de nuestro profesor, decidimos trabajar con una década que se adecue a nuestras necesidades. un caso como este, los datos se trabajan mejor con valores entre 0 y 1, por lo que se opta por normalizar los datos de TMAX para poder imputar los datos nulos con mayor facilidad.

Creamos una secuencia de alrededor de 30 pares de entrenamiento donde 30 días predicen el día siguiente

```
def crear_secuencias(data, pasos=30):
    X, y = [], []
    for i in range(len(data) - pasos):
        X.append(data[i:i+pasos])
        y.append(data[i+pasos])
    return np.array(X), np.array(y)

X, y = crear_secuencias(tmax_scaled)
print(X.shape, y.shape)
```

✓ 0.0s

(2834, 30, 1) (2834, 1)

Con esto, hacemos la division de el 70/30 que se requirio para poder entrenar el modelo. con esto separamos los datos de entrenamiento y prueba para poder evaluar el modelo posteriormente.

```
split = int(len(X) * 0.7)

x_train, x_test = x[:split], x[split:]
y_train, y_test = y[:split], y[split:]

print(f"Entrenamiento: {x_train.shape}")
print(f"Prueba: {x_test.shape}")
```

✓ 0.0s

Entrenamiento: (1983, 30, 1)
Prueba: (851, 30, 1)

FASE 3- CONSTRUIR EL MODELO

Para que este modelo pueda ser contruido, optamos por utilizar LSTM, que por sus siglas, significa Long Short-Term Memory. El clima tiene patrones temporales muy claros, como por ejemplo: La temperatura sube en verano y baja en invierno cada año, si llovió mucho esta semana, probablemente el suelo está saturado o las sequías y lluvias siguen ciclos multianuales

Una red normal no puede aprender eso. LSTM sí, porque recuerda lo que pasó antes.

Layer (type)	Output Shape	Param #
lstm_6 (LSTM)	(None, 30, 64)	16,896
dropout_6 (Dropout)	(None, 30, 64)	0
lstm_7 (LSTM)	(None, 32)	12,416
dropout_7 (Dropout)	(None, 32)	0
dense_3 (Dense)	(None, 1)	33

Total params: 29,345 (114.63 KB)

Trainable params: 29,345 (114.63 KB)

Non-trainable params: 0 (0.00 B)

FASE 4 – ENTRENAR EL MODELO

Para esta fase entrenaremos al modelo con un intervalo de 50 epocas en las que podra reconocer o buscar valores perdidos, y asi poder entenderlos a la perfeccion

```
history = model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.1,
    verbose=1
)
```

✓ 1m 11.2s

Epoch	56/56	Time	loss	mae	val_loss	val_mae
Epoch 1/50	8s	42ms/step	0.0435	0.1532	0.0082	0.0688
Epoch 2/50	1s	22ms/step	0.0185	0.1053	0.0086	0.0727
Epoch 3/50	1s	21ms/step	0.0184	0.1054	0.0085	0.0723
Epoch 4/50	1s	23ms/step	0.0169	0.0997	0.0094	0.0780
Epoch 5/50	1s	23ms/step	0.0169	0.1001	0.0082	0.0705
Epoch 6/50	1s	22ms/step	0.0169	0.0996	0.0081	0.0702

FASE 5- EVALUAR EL MODELO

Para continuar con la siguiente fase, necesitaremos evaluar el modelo utilizando las metricas para modelos de regresion, los cuales seran tanto el MSE y el R2, incluyendo tambien al MAE para tener una mejor idea de la precision del modelo.

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# Hacer predicciones
y_pred = model.predict(X_test)

# Desnormalizar los valores
y_test_real = scaler.inverse_transform(y_test)
y_pred_real = scaler.inverse_transform(y_pred)

# Calcular métricas
mse = mean_squared_error(y_test_real, y_pred_real)
mae = mean_absolute_error(y_test_real, y_pred_real)
r2 = r2_score(y_test_real, y_pred_real)

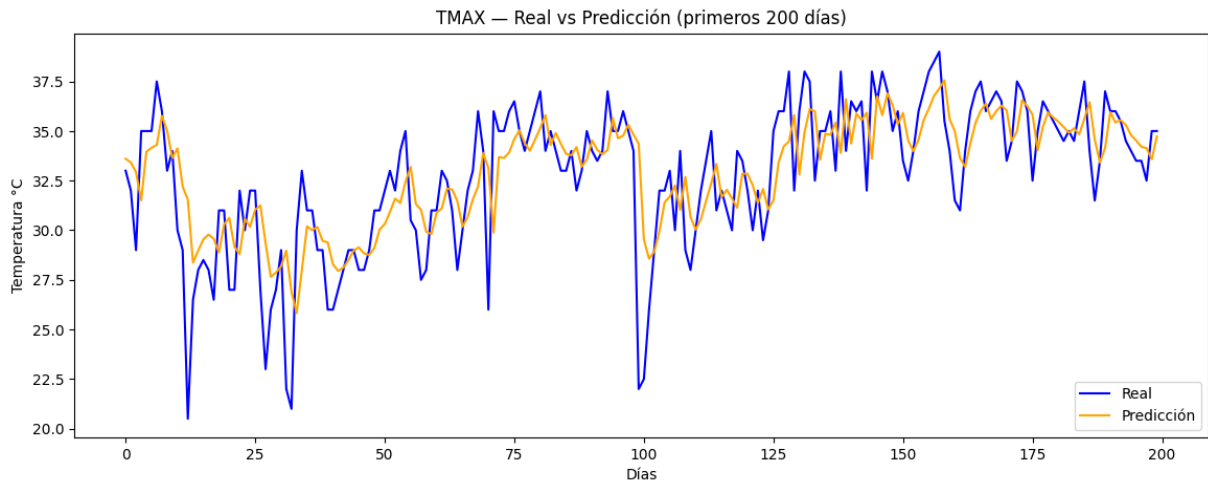
print(f"MSE: {mse:.4f}")
print(f"MAE: {mae:.4f}")
print(f"R²: {r2:.4f}")
```

✓ 1.6s

27/27 — 1s 36ms/step

MSE: 4.2396
MAE: 1.4786
R²: 0.5007

Despues de una prediccion de 200 dias este fue el resultado que nos arroja a partir de las epocas entrenadas



FASE FINAL – IMPUTAR LOS DATOS

Para esto simulamos que se obtuvieran 2 años completos, los cuales seran 2015 y 2016, los cuales con sus registros podremos obtener algo de donde imputarlos.

En donde a la hora de aplicar el modelo, nos arroja una grafica que nos explica que los registros quedan un tanto alejados, ya que presentan muy pocos datos con los cuales pudo ser evaluado. Agregamos la variable del mes para que pudiera entender que a lo largo de los meses la temperatura puede llegar a variar, dandonos el resultado anexo a continuacion

